

CIS-11 Course Project Documentation

Project B: Test Score Calculator

Team Name: Avengers Assemble

Team Members: David Zaragoza, Javier Martinez, Francisco Colorado

Project Name: Test Score Calculator

Date: May 24, 2025

Advisor: Kasey Nguyen, PhD

Part I - Application Overview

This requirements document presents the full scope of the Test Score Calculator, an LC-3 assembly language program developed as part of the CIS-11 course project. The program accepts five integer test scores from the user, computes the minimum, maximum, and average, and displays the letter grade associated with each value.

Objectives

Why are we doing this?

The primary objective of this project is to apply core computer architecture and assembly programming concepts taught in CIS-11 by building a working LC-3 program. The program solves a practical academic problem: summarizing a set of test scores into meaningful statistics that a student or instructor would find useful.

What academic objectives does this project help achieve?

- The project reinforces understanding of LC-3 instruction set architecture, including arithmetic, data movement, and conditional branch instructions.
- It demonstrates the practical use of subroutines, stack management (PUSH/POP), and save-restore patterns that mirror real-world calling conventions.
- It practices ASCII conversion for numeric output, which is a fundamental skill in low-level programming.

Why are we doing this project now?

This project is assigned at a point in the course when students have covered all foundational LC-3 concepts. Completing it now allows students to consolidate that knowledge into a single cohesive program before the course ends. Delaying would reduce time available for testing and

refinement. Not doing it at all would leave a gap between theoretical instruction and practical application.

Who will benefit from this project?

- Students enrolled in CIS-11 will benefit by gaining hands-on experience writing, testing, and debugging LC-3 assembly code.
- Instructors benefit by having a concrete deliverable to assess student mastery of course concepts.
- Any future student reviewing this documentation will benefit from seeing a complete, well-documented example of an LC-3 program.

Business Process

The Test Score Calculator addresses a common academic workflow: a student or instructor needs a quick summary of a set of exam scores. Currently, this would be done manually or with a spreadsheet. The LC-3 program replaces that process in a low-level computing environment, demonstrating that even a minimal instruction set can handle real data processing tasks.

The process flow with the new system is as follows:

- The user launches the program in the LC-3 simulator.
- The program displays a welcome message and prompts the user to enter five test scores one at a time.
- Each score is read from the keyboard, converted from ASCII input, and stored in a memory array.
- After all five scores are collected, the program calls subroutines to find the minimum, maximum, and average.
- The program calls a grading subroutine for each computed value and displays the results with labels and letter grades on the console.
- The program halts cleanly.

User Roles and Responsibilities

Role 1: End User (Student or Instructor)

- Runs the program in the LC-3 simulator.
- Enters five integer test scores when prompted (one per prompt, values 0 to 100).
- Reads the output: minimum score with grade, maximum score with grade, and average score with grade.
- Uses the displayed results to interpret overall performance on a set of exams.

Role 2: Developer (Project Team)

- Designs the program architecture, including memory layout and subroutine structure.
- Writes and debugs the LC-3 assembly code.
- Test the program using the required test values (52, 87, 96, 79, 61) and verifies correct output.
- Completes and submits all project documentation for each part.

Production Rollout Considerations

This program is deployed and run inside the LC-3 simulator environment (LC3Tools or equivalent). Rollout consists of the following steps:

- Load the assembled .obj file into the LC-3 simulator.
- Set the program counter to the origination address (x3000).
- Run the simulation and verify that the console correctly prompts for five scores.
- Enter the test values (52, 87, 96, 79, 61) and confirm output matches expected results: Min = 52 (F), Max = 96 (A), Avg = 75 (C).
- Verify that no memory overflow occurs and the stack pointer is correctly restored after each subroutine call.

No external database or network is involved. All data is held in LC-3 memory during a single execution session and is not persisted after the program halts.

Terminology

- LC-3: Little Computer 3, a simple 16-bit educational computer architecture used to teach assembly language programming.
- Assembly Language: A low-level programming language with a strong correspondence to machine code instructions.
- Subroutine: A reusable block of assembly code invoked with JSR (Jump to Subroutine) and returned from with RET.
- Stack: A region of memory managed in last-in-first-out (LIFO) order using a stack pointer (R6). PUSH decrements R6 and stores a value; POP loads a value and increments R6.
- Save-Restore: A convention where a subroutine saves all registers it uses onto the stack at entry and restores them before returning, preserving caller state.
- ASCII Conversion: Translating a binary integer to its printable ASCII character representation (e.g., adding x30 converts digit 0-9 to ASCII '0'-'9').
- Minimum: The smallest value in the set of input scores.

- **Maximum:** The largest value in the set of input scores.
- **Average:** The arithmetic mean of the five scores, computed as the sum divided by 5.
- **Letter Grade:** A letter (A through F) assigned to a numeric score based on the grading scale defined in the project requirements.
- **Pointer:** A register that holds a memory address, used to traverse the scores array element by element.
- **TRAP:** An LC-3 system call directive (e.g., TRAP x21 prints a character, TRAP x22 prints a string, TRAP x25 halts the program).

Part II - Functional Requirements

Statement of Functionality

The Test Score Calculator shall perform the following functions:

- Display a welcome message to the console using TRAP x22 when the program starts.
- Prompt the user five times to enter an integer test score (0 to 100) using TRAP x22.
- Read each score from the keyboard using TRAP x23, convert from ASCII input to a binary integer, and store it in a pre-allocated memory array (SCORES) using a pointer register.
- After all five scores are stored, call subroutine FIND_MIN_MAX, which iterates the array using a pointer and returns the minimum value in R3 and the maximum value in R4.
- Call subroutine CALC_AVERAGE, which sums the five array values and divides the sum by 5 using iterative subtraction, returning the average in R5.
- Call subroutine GET_GRADE for the minimum, maximum, and average scores. GET_GRADE accepts a score in R0, compares it against the grade thresholds using conditional branches, and returns the ASCII code of the appropriate letter grade in R0.
- Convert the binary values of minimum, maximum, and average to ASCII digit sequences using addition of x30 per digit for console display.
- Display labeled output lines on the console: "Min: [value] [grade]", "Max: [value] [grade]", and "Avg: [value] [grade]".
- Use at least two subroutines (FIND_MIN_MAX and GET_GRADE, with CALC_AVERAGE as a third).
- Include PUSH and POP stack operations within each subroutine for save-restore of all modified registers.
- Use a pointer register (R1) to traverse the SCORES array in all loops.
- Use appropriate LC-3 directives: .ORIG x3000, .FILL, .BLKW for array allocation, and .END.
- Halt execution cleanly with TRAP x25.

Grading scale used by GET_GRADE:

Score Range	Letter Grade
90 to 100	A
80 to 89	B
70 to 79	C
60 to 69	D
0 to 50	F

Scope

Phase	Deliverable	Functionality Included
Part 1	Project Documentation	Objectives, pseudocode, flowchart, team tasks
Part 2	Refined Design	Updated pseudocode, detailed memory map, subroutine interface specs
Part 3	Final LC-3 Program + Testing	Full assembly source, tested with values 52, 87, 96, 79, 61; verified output

Performance

- The program shall accept all five scores and display all results within 500 LC-3 clock cycles per test score entered, for a total of no more than 3,000 cycles end to end on the LC3Tools simulator running at default speed.
- Each subroutine (FIND_MIN_MAX, CALC_AVERAGE, GET_GRADE) shall execute in no more than 100 instructions per call.
- The CALC_AVERAGE division loop shall iterate at most 100 times (sufficient for a maximum score sum of 500 divided by 5).
- Stack depth shall not exceed 10 words at any point during execution, keeping memory usage within the allocated stack region.

Usability

- Each input prompt shall include the score number so the user knows which score they are entering (e.g., "Enter score 1:", "Enter score 2:").

- All output lines shall be clearly labeled so results are unambiguous (e.g., "Min: 52 Grade: F").
- A user unfamiliar with LC-3 shall be able to operate the program by following on-screen prompts without any external instructions.
- The program shall display a newline between each output line so results are not crowded together on a single console line.

Requests for Enhancements

Date	Enhancement	Requested by	Notes	Priority	Release / Status
05/20/2025	Support input validation to reject scores outside 0 to 100 and re-prompt the user	David Zaragoza	Requires additional branch logic and error message string	Medium	Part 3 / Planned
05/20/2025	Display individual letter grade for each of the five entered scores, not just min/max/avg	Javier Martinez	Requires loop over array after grading pass; additional output strings needed	Low	Part 3 / Under Review
05/20/2025	Allow user to re-run the program for a new set of scores without restarting the simulator	Francisco Colorado	Would require a restart loop and clearing the SCORES array before re-entry	Low	Future / Backlog

Part III - Appendices

Appendix A: Program Objectives (Ordered Task List)

The following ordered list describes every task the program performs, in execution order:

1. Print welcome message to console (TRAP x22).
2. Initialize stack pointer R6, array pointer R1 pointing to SCORES, loop counter R2 = 5.
3. Prompt user: print "Enter score N:" (TRAP x22).
4. Read score via TRAP x23 (keyboard input); call READ_NUMBER to collect multi-digit input and convert ASCII digits to binary integer.
5. Store score at memory location pointed to by R1; increment R1; decrement R2. Branch back to step 3 if R2 != 0.

6. Call FIND_MIN_MAX: iterate array with pointer, compare each element, track minimum in R3 and maximum in R4. PUSH/POP registers for save-restore.
7. Call CALC_AVERAGE: sum all five array values; divide sum by 5 via iterative subtraction; store result in R5. PUSH/POP registers.
8. Call GET_GRADE(R3): compare minimum score against grade thresholds using conditional branches; return ASCII letter grade in R0.
9. Convert R3 to ASCII decimal digits; print "Min: [value] Grade: [letter]" to console.
10. Repeat steps 8 to 9 for maximum (R4) and average (R5).
11. Halt program with TRAP x25.

Appendix B: Pseudocode

Main Program

Save return address

Display welcome message

Set pointer to SCORES

input counter = 5

While input counter > 0, get user input

 Display input prompt

 Call READ_NUMBER subroutine ;This will read and convert input to integer

 Store integer into SCORES

 Advance SCORES array pointer

 input counter = input counter - 1

Call MIN_MAX subroutine ;This will find the min and max score in scores

Call CALC_AVERAGE subroutine ;This will calculate the average score

Display minimum score

Display letter grade for minimum score

Display maximum score

Display letter grade for maximum score

Display average score

Display letter grade for average score

End program

Subroutine: READ_NUMBER

Save registers

current number = 0

Read character input from user

While character is not ENTER

 Convert ASCII character to integer

 current number = current number * 10

 current number = current number + integer

 Read next character

Restore registers

Return current number

Subroutine: FIND_MIN_MAX

Save registers

Set pointer to SCORES

min = first score

max = first score

Move pointer to next score

For each remaining score in SCORES

 current = current score

 if current < min

 min = current

 if current > max

 max = current

 move to next score

Restore registers

Return

Subroutine: CALC_AVERAGE

Save registers

Set pointer to SCORES

sum = 0


```

sum counter = 5
While sum counter > 0
    Load current score
    sum = sum + current score
    Move to next score in SCORES
    sum counter = sum counter - 1
average = 0
While sum >= 5
    sum = sum - 5
    average = average + 1
Restore registers
Return average

```

Subroutine: GET_GRADE

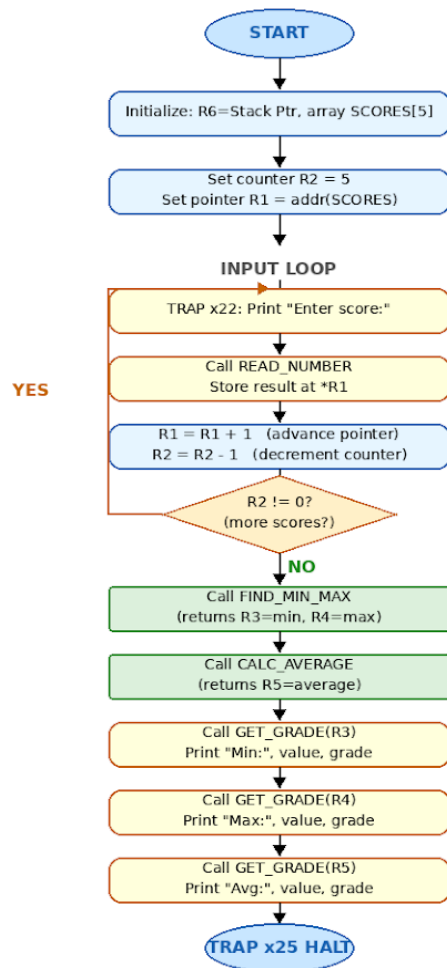
```

Save registers
If score >= 90
    letter grade = A
Else if score >= 80
    letter grade = B
Else if score >= 70
    letter grade = C
Else if score >= 60
    letter grade = D
Else
    letter grade = F
Restore Registers
Return letter grade

```

Appendix C: Program Flowchart

The flowchart below illustrates the complete control flow of the Test Score Calculator, including the input loop, subroutine calls, conditional branching inside GET_GRADE, and the output sequence.



Appendix D: Team Task Assignment

Team Member	Assigned Task	Deadline	Status
David Zaragoza	Project documentation, pseudocode, flowchart (Part 1)	May 24, 2025	Completed
Francisco Colorado	LC-3 code: input/output routines, READ_NUMBER, PRINT_NUMBER, TRAP calls (Part 3)	June 10, 2025	Not Started
Javier Martinez	LC-3 code: FIND_MIN_MAX, CALC_AVERAGE, GET_GRADE subroutines and testing (Part 3)	June 10, 2025	Not Started